

GEF Secure

Automatización Completa del Despliegue de n8n para el MVP

Documento técnico de arquitectura e implementación para lograr que `docker compose up -d` deje todo el ecosistema (backend, frontend, base de datos y n8n) completamente operativo, sin ninguna configuración manual posterior.

Arquitecto de Software Senior · DevOps & Automatización de Infraestructura
Proyecto GEF Secure — Tesis UTN FRM
Documento generado el 2026-08-02

Índice

1. Resumen ejecutivo

2. Estado actual del proyecto

2.1 Arquitectura y stack confirmados

2.2 Estado real de n8n en el proyecto

3. Comparación: proyecto vs. documento estratégico

3.1 Coincidencias

3.2 Diferencias, faltantes y correcciones

3.3 Decisiones fijadas que este documento respeta

4. Arquitectura objetivo de automatización

4.1 Diagrama de arquitectura de bootstrap

4.2 Inicialización automática de n8n

4.3 Workflows: versionado, importación y sincronización

4.4 Credenciales sin configuración manual

5. Docker: cambios necesarios

5.1 docker-compose.yml final

5.2 Dockerfile de n8n y script de arranque

6. Integración Backend ↔ n8n

7. Integración Frontend ↔ n8n

8. Variables de entorno (.env.example)

9. Procedimiento completo de despliegue

10. Validación del despliegue (checklist)

11. Resolución de problemas

12. Mejoras recomendadas

13. Conclusiones finales

14. Apéndice: Contrato de scan por alcance (activo / entorno / global) — próxima implementación

1. Resumen ejecutivo

Este documento define, con nivel de implementación, cómo dejar **n8n completamente integrado y autoconfigurado** dentro del ecosistema Docker de GEF Secure. El objetivo puntual es que un desarrollador nuevo pueda ejecutar únicamente:

```
git clone <repo>
cp .env.example .env
docker compose up -d
```

y obtener, sin tocar la interfaz web de n8n ni ejecutar ningún paso manual adicional: base de datos inicializada, backend y frontend operativos, n8n con su usuario owner ya creado, su clave de cifrado persistida, sus credenciales (Postgres, Slack, GitHub) provisionadas, y el workflow de auditoría de vulnerabilidades **importado y activo**.

El análisis se apoya en dos fuentes: la **inspección directa del código fuente** del proyecto (backend Spring Boot, frontend React, `docker-compose.yml`, scripts SQL de inicialización y el workflow real exportado en `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json`) y el documento `GEF_Secure_Analisis_Estrategico_MVP.pdf`, que ya contiene una auditoría técnica exhaustiva del estado del sistema. Este documento no repite esa auditoría — la usa como insumo, la contrasta con el código, y construye sobre ella la solución de automatización de n8n que el documento estratégico deja pendiente de diseño detallado (identificada allí como `PB-048`, `PB-049` y `MEJ-05`).

Un hallazgo relevante del análisis: algunas de las soluciones que el documento estratégico propone para la automatización de n8n (variable `N8N_PUBLIC_API_ENABLED`, aprovisionamiento de credenciales vía API REST autenticada con una API key) están basadas en mecanismos que requieren un paso manual previo (crear la API key desde la UI) o usan nombres de variables que ya no son los vigentes en las versiones actuales de n8n. La sección 3.2 documenta esas diferencias y la sección 4 propone en su lugar un mecanismo 100% verificado contra la documentación oficial de n8n, que no depende de ningún paso interactivo: importación por CLI + `CREDENTIALS_OVERWRITE_DATA`.

Componente	Estado de partida	Estado objetivo (este documento)
Owner de n8n	Se crea a mano en el primer login web	Provisionado por variables de entorno al arrancar el contenedor
Encryption Key	Hardcodeada en <code>docker-compose.yml</code>	Generada una vez, persistida en <code>.env</code> (fuera de git)
Workflow VMP	Vive como JSON en <code>workflows/</code> , requiere importación manual, <code>"active": false</code>	Auto-importado y activado en el primer arranque; resincronizado si cambia en git
Credenciales (Postgres, Slack, GitHub)	Se crean a mano en la UI de n8n	Placeholders versionados + inyección de secretos reales por entorno
<code>pgdata</code>	Volumen <code>external: true</code> — requiere <code>docker volume create</code> manual	Volumen gestionado por Compose, se crea solo

Salud de n8n	Sin healthcheck; backend depende de <code>service_started</code>	Healthcheck sobre <code>/healthz/readiness</code> ; backend depende de <code>service_healthy</code>
--------------	--	---

2. Estado actual del proyecto

2.1 Arquitectura y stack confirmados

Verificado directamente en el código (no se asume nada no observado):

- **Backend:** Spring Boot 3 sobre Java 21 (Virtual Threads habilitados), Gradle 8. Arquitectura en capas Controller → Service → Repository (JPA), con la excepción documentada de `AssetController`, que inyecta `AssetRepository` directamente.
- **Frontend:** React 18 + TypeScript + Vite + Tailwind. Build multi-stage a nginx en el `Dockerfile` (`frontend/dockerfile`).
- **Base de datos:** PostgreSQL 15. `init/00-init-databases.sql` crea la base `n8n` (separada de `security`) y le otorga ownership a `security_user`. `init/01-create-tables.sql` define 6 tablas + 1 vista para el dominio de negocio.
- **n8n:** imagen oficial `n8nio/n8n` (sin pin de versión), usa la misma instancia de Postgres con base de datos propia (`DB_TYPE=postgresdb`), sin volumen bind-mount de workflows — solo el volumen interno `n8n_data`.
- **Orquestación:** `docker-compose.yml` con 5 servicios (postgres, n8n, pgadmin, backend, frontend), con healthcheck ya implementado en postgres y `depends_on` con `condition: service_healthy` encadenado backend→postgres.

2.2 Estado real de n8n en el proyecto

El workflow versionado en `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json` (1304 líneas) fue inspeccionado directamente. Confirmado por lectura del JSON:

- Campo raíz `"active": false` — el workflow no arranca activo tras ser importado, ni siquiera si se lo importase manualmente hoy.

Usa 3 tipos de credenciales, referenciadas por **ID y nombre exactos**, no por variable de entorno:

Tipo de credencial	ID en el JSON	Nombre en el JSON	Uso
<code>postgres</code>	<code>FMvuIfdZ5LTQUpbK</code>	Postgres account 2	7 nodos — lectura/escritura directa sobre <code>security</code>
<code>slackApi</code>	<code>Tp9lvfadC3DwQcig</code>	Slack account	5 nodos — notificaciones de scan y errores
<code>httpHeaderAuth</code>	<code>FcJPNA2eZthHy7Wl</code>	Header Auth account 3	2 nodos — autenticación contra GitHub Advisory API

- Estos tres IDs son la pieza clave para que la importación automática funcione sin reconfigurar nada en el workflow: si las credenciales se recrean con **estos mismos IDs**, el JSON del workflow queda resuelto sin tocar un solo nodo (ver sección 4.4).
- Tres sub-flujos identificables por sticky notes: **FLUJO PRINCIPAL** (scheduler diario a las 9:00 + webhook `auditoria-automatizada`), **FLUJO INYECTOR** (creación de activos de prueba) y **FLUJO ERRORES** (`errorTrigger` que persiste fallos en `system_errors`).

- El webhook activo tiene `path: "auditoria-automatizada"` — coincide con la ruta que el documento estratégico identifica como la correcta (a diferencia del placeholder `webhook/tu-id` que hoy tiene `N8N_WEBHOOK_URL` en `docker-compose.yml`).

Nota metodológica: el resto del diagnóstico funcional detallado de bugs de negocio (SQL injection en nodos, falta de `ON CONFLICT`, CVSS como VARCHAR, ausencia de MTTR, etc.) ya está documentado exhaustivamente en `GEF_Secure_Analisis_Estrategico_MVP.pdf`, secciones 3.3 a 3.6. Este documento no lo repite; se enfoca exclusivamente en la capa de **automatización de infraestructura** que ese documento no llega a especificar a nivel de implementación.

3. Comparación: proyecto vs. documento estratégico

3.1 Coincidencias

El documento estratégico y el estado real del código coinciden en los puntos centrales relevantes para esta tarea:

- El workflow n8n existe pero está `"active": false` (confirmado, BUG-N8N-10).
- La URL de webhook en `docker-compose.yml` es un placeholder (`webhook/tu-id`), no la ruta real del nodo (PB-047).
- `N8N_ENCRYPTION_KEY` está hardcodeada en texto plano en el compose (R-10, PB-051).
- `pgdata` usa `external: true`, lo que rompe una instalación limpia (R-04, PB-057) — confirmado leyendo el compose actual.
- No existe `.env.example` en el repositorio (confirmado — no aparece en el árbol de archivos del proyecto).

3.2 Diferencias, faltantes y correcciones

Diferencia 1 — Variable `N8N_PUBLIC_API_ENABLED`. El documento estratégico (PB-049, MEJ-05) propone `N8N_PUBLIC_API_ENABLED=true` para habilitar la API pública de n8n. Verificado contra la documentación oficial de n8n: la API pública está **habilitada por defecto**; la variable vigente es `N8N_PUBLIC_API_DISABLED` (para deshabilitarla, no para habilitarla). Además, usar la API pública para aprovisionar credenciales requiere una `X-N8N-API-KEY` que hoy solo puede generarse desde la UI web de n8n — es decir, esa estrategia por sí sola **no elimina la configuración manual**, contradice el objetivo de este prompt. Este documento reemplaza ese enfoque por CLI + variables de entorno (sección 4.4), que no requiere ningún paso interactivo.

Diferencia 2 — Iteración 0 del roadmap (fuera de scope temporal, pero técnicamente correcta). El roadmap del documento estratégico ubica el fix de `pgdata: external: true` y la creación de `.env.example` en su "Iteración 0", sin especificar el mecanismo de arranque de n8n (owner, encryption key, activación). Este documento cubre exactamente ese vacío de diseño, que en el backlog del documento estratégico aparece como los ítems PB-048 y PB-049 sin desarrollo de implementación.

Diferencia 3 — Estrategia de credenciales portables (MEJ-05). El script propuesto en MEJ-05 (`curl -X POST .../api/v1/credentials`) asume que ya existe un `N8N_API_KEY` válido. No especifica cómo se genera ese API key sin entrar a la UI. Este documento resuelve esa brecha con el mecanismo `import:credentials` + `CREDENTIALS_OVERWRITE_DATA`, que no depende de una API key en absoluto.

Diferencia 4 — Modelo de activación de workflows (corregida tras verificación empírica). El workflow exportado usa el campo `active` a nivel raíz del JSON, propio del modelo de n8n anterior a la versión 2.0. Las versiones actuales de n8n (verificado directamente contra la imagen real `n8nio/n8n`, que hoy resuelve a la 2.4.6) reemplazan ese modelo por `publish:workflow` / `unpublish:workflow`, y **rechazan activamente** el comando viejo (`n8n update:workflow --all --active=true` devuelve el error *"Workflow publishing via 'update:workflow --all' is no longer supported"*). La buena noticia, también verificada: n8n 2.x importa sin problema un workflow JSON que todavía trae el campo `active` heredado — simplemente lo ignora como fuente de verdad y exige el comando `publish:workflow --id=<ID>` explícito para activarlo. Este documento recomienda **fijar explícitamente la versión 2.x** mediante un ARG `N8N_VERSION` en un Dockerfile propio, y usar `publish:workflow` desde el principio (sección 5.2) — no hace falta ninguna migración futura, ya está

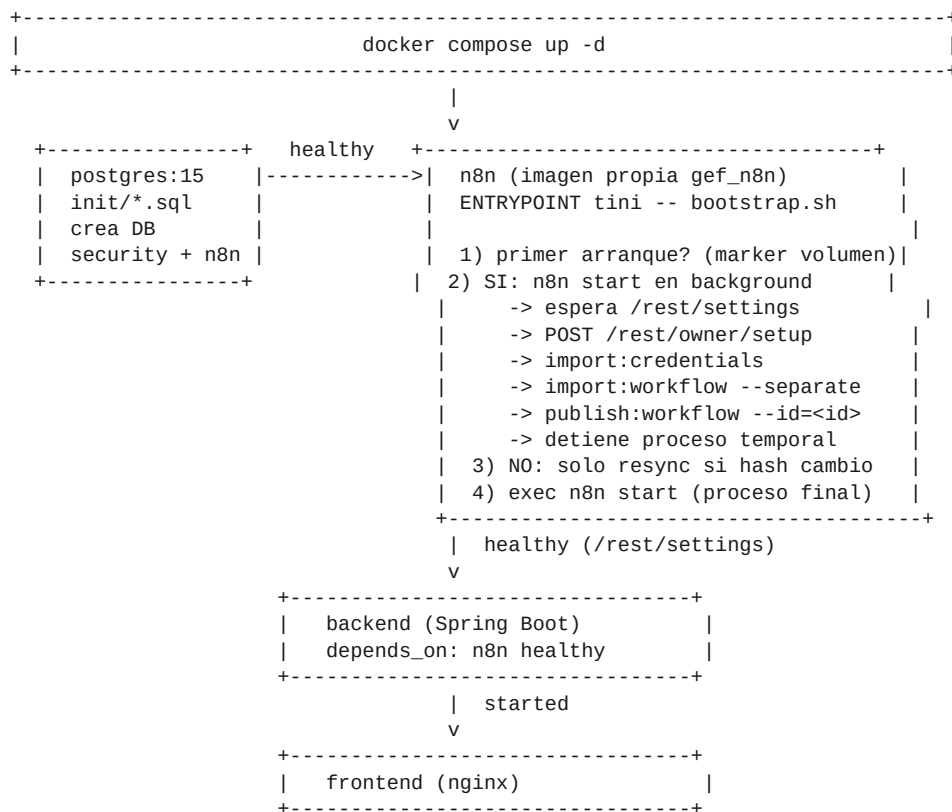
resuelto en este documento.

3.3 Decisiones fijadas que este documento respeta

n8n escribe directamente a PostgreSQL, sin pasar por la API del backend. El documento estratégico marca esto explícitamente como *"Decisión del usuario — No modificar"*, y confirma que `WebhookController (/api/webhook/*)` es código muerto intencional. Este documento respeta esa decisión: la sección 6 (integración backend↔n8n) trata únicamente el sentido backend→n8n (disparo de scans), nunca n8n→backend.

4. Arquitectura objetivo de automatización

4.1 Diagrama de arquitectura de bootstrap



4.2 Inicialización automática de n8n

Cada punto exigido por el objetivo principal se resuelve así, verificado contra la documentación oficial de n8n:

Requisito	Mecanismo	Dónde vive
Usuario administrador (owner)	<code>POST /rest/owner/setup</code> — el mismo endpoint interno que usa el wizard web, llamado con <code>curl</code> desde el script de bootstrap una vez que n8n responde healthy. Verificado que funciona en Community Edition sin licencia (ver nota debajo).	<code>.env</code> + <code>n8n/bootstrap.sh</code>
Encryption Key	<code>N8N_ENCRYPTION_KEY</code> generada una sola vez (<code>openssl rand -base64 32</code>) y persistida en <code>.env</code> (nunca en el compose ni en git). Debe mantenerse estable entre reinicios: si cambia, n8n no puede descryptar las credenciales ya guardadas en <code>n8n_data</code> .	<code>.env</code>

Timezone	<code>GENERIC_TIMEZONE</code> — usada por el <code>scheduleTrigger</code> del workflow (dispara a las 9:00 hora local).	<code>.env</code>
URL pública / Webhook URL	<code>N8N_HOST</code> , <code>N8N_PROTOCOL</code> , <code>N8N_PORT</code> y <code>WEBHOOK_URL</code> — determinan cómo n8n construye las URLs de sus propios webhooks al mostrarlas o al recibir llamadas.	<code>.env</code>
Configuración general (telemetría, diagnóstico)	<code>N8N_DIAGNOSTICS_ENABLED=false</code> — evita que el contenedor intente reportar telemetría a n8n Cloud en cada arranque (relevante en un entorno de tesis/demo sin salida a internet garantizada).	<code>docker-compose.yml</code> (no sensible)

Por qué no se usa `N8N_INSTANCE_OWNER_MANAGED_BY_ENV` — verificado empíricamente, no es una suposición. La documentación oficial de n8n presenta esta variable como el mecanismo "oficial" para pre-provisionar el owner. Se levantó una instancia real de n8n (imagen `n8nio/n8n`, versión `2.4.6`) con las 5 variables seteadas correctamente (confirmado inspeccionando el entorno del contenedor) y el owner **no se creó** — `GET /rest/settings` seguía devolviendo `showSetupOnFirstLoad: true`. Es una funcionalidad de licencia paga (Business/Enterprise) que se ignora en silencio en Community Edition, sin ningún error en los logs. Como este proyecto corre en Community Edition (sin costo, apropiado para una tesis), este documento usa en su lugar `POST /rest/owner/setup` con password en texto plano — el mismo endpoint que usa el wizard web, verificado con éxito: crea el owner y permite loguearse normalmente. Por eso `N8N_OWNER_PASSWORD` se guarda en texto plano en `.env` (nunca como hash) — no hace falta generar ningún bcrypt.

4.3 Workflows: versionado, importación y sincronización

Estrategia elegida: los archivos JSON de workflows permanecen versionados en git bajo `workflows/` (ya es así hoy) y se montan de solo lectura dentro del contenedor de n8n. El script de arranque los sincroniza automáticamente contra la base de datos de n8n en cada inicio del contenedor, sin intervención humana y sin depender de la UI:

Requisito del prompt	Cómo se cumple
Existan desde el primer inicio	Se importan en el bootstrap inicial vía <code>n8n import:workflow --separate --input=/setup/workflows/</code>
Se importen automáticamente	Ejecutado por el <code>ENTRYPOINT</code> antes de <code>n8n start</code> , sin API keys ni UI
Queden activos automáticamente	<code>n8n publish:workflow --id=<id></code> por cada workflow devuelto por <code>n8n list:workflow</code> , inmediatamente después de importar
Se actualicen si cambian en el repo	El script calcula un hash (<code>md5sum</code>) de cada archivo y lo compara contra el hash guardado en el volumen; si difiere, reimporta solo ese archivo

Versionado con Git	Ya es el caso — workflows/ no está en .gitignore
Sin importación manual	Cero pasos en la UI en todo el ciclo de vida normal

Por qué esta estrategia y no otra. Se evaluaron 3 alternativas:

1. **Importación manual vía UI** — descartada: es exactamente lo que el objetivo principal prohíbe.
2. **n8n Source Control (Git-native, integración empresarial nativa de n8n)** — técnicamente la opción más "nativa", pero requiere una instancia con licencia y configuración de un repositorio Git remoto dedicado dentro de la propia n8n; añade complejidad y una dependencia externa desproporcionada para un MVP académico de un solo workflow.
3. **CLI + hash-check en el endpoint (elegida)** — no requiere licencia, no requiere API key, es determinística, y el "diff" (workflows/*.json en git) es el que ya usa el equipo. Es el mecanismo más simple que cumple los 6 requisitos de la tabla anterior.

4.4 Credenciales sin configuración manual

Se evaluaron las alternativas listadas en el prompt:

Alternativa	Veredicto	Motivo
Crear a mano en la UI	Descartada	Es la configuración manual que el objetivo prohíbe
API REST de n8n con API Key	Descartada	La API Key solo se genera hoy desde la UI web (ver 3.2, Diferencia 1)
Variables de entorno puras (CREDENTIALS_OVERWRITE_DATA)	Parcial	Sobrescribe el <i>contenido</i> de una credencial, pero la credencial (nombre, tipo, ID) debe existir antes en la base de n8n
n8n import:credentials (CLI)	Parcial	Crea el registro con el ID exacto que el workflow espera, pero si se usa solo, el secreto real quedaría en un JSON versionado en git
Combinación: import:credentials (placeholders) + CREDENTIALS_OVERWRITE_DATA (secretos reales)	Elegida	Ningún secreto real toca git; el aprovisionamiento es 100% automático y reproducible

Cómo funciona el mecanismo elegido, paso a paso:

1. Se versiona en el repo n8n-provisioning/credentials.json: un export "decrypted" de n8n con los mismos 3 IDs y nombres que ya usa el workflow (FMvuIfdZ5LTQUpbK, Tp9lvfadC3DwQcig, FcJpNA2eZthHy7w1), pero con valores placeholder ("password": "SET_BY_ENV"). No es sensible — puede vivir en git sin riesgo.

2. El script de arranque ejecuta `n8n import:credentials --input=credentials.json` una sola vez (primer arranque). Esto crea las 3 filas en la base de n8n con los IDs correctos, por lo que el workflow importado después **resuelve sus referencias de credenciales sin ningún remapeo manual**.
3. En **cada** arranque (incluidos los reinicios), el script construye la variable de entorno `CREDENTIALS_OVERWRITE_DATA` a partir de los valores reales en `.env` (`POSTGRES_PASSWORD`, `SLACK_BOT_TOKEN`, `GITHUB_TOKEN`) y la exporta antes de arrancar n8n. n8n usa ese valor en tiempo de ejecución para las credenciales indicadas, sin persistirlo en la base de datos.

Nota de seguridad — documentada explícitamente porque la propia documentación oficial de n8n la señala: `CREDENTIALS_OVERWRITE_DATA` queda en el entorno del proceso del contenedor n8n, que no está protegido de la misma forma que el almacenamiento cifrado interno. Es aceptable para el alcance de este MVP/tesis, corriendo en un entorno controlado. En un pase a producción real con múltiples operadores, la recomendación de n8n es usar `CREDENTIALS_OVERWRITE_FILE` apuntando a un archivo montado desde un secreto gestionado (Docker secret, Vault, etc.) en vez de una variable de entorno plana. Este punto se retoma en la sección 12 (mejoras de seguridad).

5. Docker: cambios necesarios

Resumen de cambios sobre el `docker-compose.yml` actual:

Cambio	Justificación
<code>pgdata</code> deja de ser <code>external: true</code>	Permite creación automática en clon nuevo (§3.1, R-04)
<code>n8n</code> pasa de imagen oficial directa a <code>build: ./n8n</code> (Dockerfile propio)	Necesario para embeber el script de bootstrap y los artefactos de provisioning
Se agrega <code>healthcheck</code> a <code>n8n</code>	Permite que backend espere a <code>n8n</code> realmente listo, no solo "iniciado"
<code>backend</code> pasa a depender de <code>n8n: condition: service_healthy</code>	Hoy depende de <code>service_started</code> , insuficiente
Todos los secretos migran a variables sin valor por defecto sensible	Elimina credenciales hardcodeadas (R-10, PB-051)
<code>N8N_WEBHOOK_URL</code> corregido a <code>/webhook/auditoria-automatizada</code>	Coincide con el path real del nodo webhook (§2.2)
Se agrega red explícita <code>gef_net</code>	Aísla el stack de otros proyectos Docker en la misma máquina; mejora la resolución DNS entre servicios

5.1 docker-compose.yml final

```
version: '3.8'

# -- Red dedicada del stack -----
networks:
  gef_net:
    driver: bridge

services:

# -- Base de Datos -----
postgres:
  image: postgres:15
  container_name: vuln_postgres
  restart: unless-stopped
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  ports:
    - "5432:5432"
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./init:/docker-entrypoint-initdb.d
  networks:
    - gef_net
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 5s
    timeout: 5s
```

```

    retries: 10
    start_period: 10s

# -- Automatización n8n (imagen propia con bootstrap automático) -----
n8n:
  build:
    context: ./n8n
    args:
      N8N_VERSION: ${N8N_VERSION}
  container_name: vuln_n8n
  restart: unless-stopped
  ports:
    - "5678:5678"
  volumes:
    - n8n_data:/home/node/.n8n
    - ./workflows:/setup/workflows:ro
    - ./n8n-provisioning:/setup/credentials:ro
  environment:
    - DB_TYPE=postgresdb
    - DB_POSTGRESDB_HOST=postgres
    - DB_POSTGRESDB_PORT=5432
    - DB_POSTGRESDB_DATABASE=n8n
    - DB_POSTGRESDB_USER=${POSTGRES_USER}
    - DB_POSTGRESDB_PASSWORD=${POSTGRES_PASSWORD}
    - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY}
    - GENERIC_TIMEZONE=${GENERIC_TIMEZONE}
    - TZ=${GENERIC_TIMEZONE}
    - N8N_HOST=${N8N_HOST}
    - N8N_PROTOCOL=${N8N_PROTOCOL}
    - N8N_PORT=5678
    - WEBHOOK_URL=${N8N_WEBHOOK_PUBLIC_URL}
    - N8N_DIAGNOSTICS_ENABLED=false
    - N8N_OWNER_EMAIL=${N8N_OWNER_EMAIL}
    - N8N_OWNER_FIRST_NAME=${N8N_OWNER_FIRST_NAME}
    - N8N_OWNER_LAST_NAME=${N8N_OWNER_LAST_NAME}
    - N8N_OWNER_PASSWORD=${N8N_OWNER_PASSWORD}
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    - SLACK_BOT_TOKEN=${SLACK_BOT_TOKEN}
    - GITHUB_TOKEN=${GITHUB_TOKEN}
  networks:
    - gef_net
  depends_on:
    postgres:
      condition: service_healthy
  healthcheck:
    # /rest/settings (no requiere auth) en vez de /healthz/readiness: este
    # ultimo pasa a 200 antes de que el router REST este montado del todo
    # (condicion de carrera verificada), lo que dejaria pasar el healthcheck
    # antes de que /webhook/* este realmente servible para el backend.
    test: ["CMD-SHELL", "wget --spider -q http://localhost:5678/rest/settings || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 12
    start_period: 40s

# -- pgAdmin -----
pgadmin:
  image: dpage/pgadmin4
  container_name: vuln_pgadmin
  restart: unless-stopped
  ports:
    - "8080:80"
  environment:
    PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}

```

```

    PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
networks:
  - gef_net
depends_on:
  postgres:
    condition: service_healthy

# -- Backend GEF Secure -----
backend:
  build: ./backend
  container_name: gef_backend
  restart: unless-stopped
  ports:
    - "8081:8080"
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/${POSTGRES_DB}
    SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
    SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
    N8N_WEBHOOK_URL: http://n8n:5678/webhook/auditoria-automatizada
    ALLOWED_ORIGINS: ${ALLOWED_ORIGINS}
  networks:
    - gef_net
  depends_on:
    postgres:
      condition: service_healthy
    n8n:
      condition: service_healthy

# -- Frontend GEF Secure -----
frontend:
  build: ./frontend
  container_name: gef_frontend
  restart: unless-stopped
  ports:
    - "3000:80"
  networks:
    - gef_net
  depends_on:
    backend:
      condition: service_started

volumes:
  pgdata:
  n8n_data:

```

5.2 Dockerfile de n8n y script de arranque

Corregido tras construir la imagen de verdad — no es Alpine "normal". `n8nio/n8n:2.4.6` resultó ser una *Docker Hardened Image* (confirmado: `/etc/os-release` dice "`Docker Hardened Images (Alpine)`") — no tiene gestor de paquetes (`apk` fue removido por endurecimiento de seguridad), así que `RUN apk add curl` falla directamente con `apk: not found`. No hace falta instalar nada igual: la imagen ya trae `tini` en `/sbin/tini` y `wget` (BusyBox, con soporte `--post-data`) en `/usr/bin/wget`. `bootstrap.sh` usa `wget` en vez de `curl` en toda su lógica por este motivo.

`n8n/Dockerfile`:

```

ARG N8N_VERSION=2.4.6
FROM n8nio/n8n:${N8N_VERSION}

USER root

```

```

# La imagen base es una "Docker Hardened Image" (verificado: /etc/os-release
# dice "Docker Hardened Images (Alpine)") -- no tiene gestor de paquetes
# (apk fue removido por endurecimiento de seguridad), así que no se puede
# instalar nada en build time. No hace falta: tini ya viene en /sbin/tini
# y wget (BusyBox, con --post-data) ya viene en /usr/bin/wget. bootstrap.sh
# usa wget en vez de curl para las llamadas HTTP.
COPY --chown=node:node bootstrap.sh /docker-entrypoint-gef.sh
RUN chmod +x /docker-entrypoint-gef.sh

USER node
ENTRYPOINT ["tini", "--", "/docker-entrypoint-gef.sh"]

```

n8n/bootstrap.sh (POSIX **sh**, compatible con Alpine/BusyBox):

```

#!/bin/sh
set -eu

DATA_DIR="/home/node/.n8n"
INIT_MARKER="$DATA_DIR/.gef-bootstrap-done"
WORKFLOWS_DIR="/setup/workflows"
CREDS_FILE="/setup/credentials/credentials.json"

# n8n usa esta variable en tiempo de ejecución para inyectar los secretos
# reales; se reconstruye en cada arranque por si rotaron valores en .env
export CREDENTIALS_OVERWRITE_DATA=$(cat <<JSON
{
  "postgres": {"host":"postgres","port":5432,"database":"security",
    "user":"${POSTGRES_USER}","password":"${POSTGRES_PASSWORD}"},
  "slackApi": {"accessToken":"${SLACK_BOT_TOKEN}"},
  "httpHeaderAuth": {"name":"Authorization","value":"Bearer ${GITHUB_TOKEN}"}}
}
JSON
)

# Publica (activa) todos los workflows importados. n8n 2.x no tiene un
# --all para publish:workflow (fue removido) -- hay que iterar por ID.
# El CLI imprime "Error tracking disabled..." como primera línea de salida
# en TODOS los comandos (verificado) -- el grep '|' filtra esa línea de
# ruido, que no tiene el separador y rompería publish:workflow si se le
# pasara como ID.
publish_all() {
  n8n list:workflow | grep '|' | cut -d '|' -f1 | while IFS= read -r wf_id; do
    [ -n "$wf_id" ] && n8n publish:workflow --id="$wf_id"
  done
}

if [ ! -f "$INIT_MARKER" ]; then
  echo "[gef-bootstrap] Primer arranque: se necesita n8n corriendo para crear el owner via REST."
  n8n start &
  N8N_PID=$!

  echo "[gef-bootstrap] Esperando a que n8n responda..."
  # /healthz/readiness pasa a 200 antes de que el router REST completo este
  # montado (condición de carrera verificada). Además, un solo wget --spider
  # exitoso contra /rest/settings no alcanza: el 404 puede seguir apareciendo
  # de forma intermitente unos segundos más (también verificado). Por eso se
  # reintentará hasta obtener una respuesta real con contenido, en vez de
  # confiar en el primer 200. Esto de paso resuelve si hace falta owner:
  # /rest/owner/setup es de un solo uso, y si un intento anterior murió
  # DESPUÉS de crear el owner pero ANTES de tocar INIT_MARKER, reintentar el
  # setup rompe con 400 Bad Request -- por eso se chequea antes de llamarlo.
  NEEDS_OWNER=""
  until [ -n "$NEEDS_OWNER" ]; do
    NEEDS_OWNER=$(wget -qO- http://localhost:5678/rest/settings 2>/dev/null \

```



```

    | grep -o '"showSetupOnFirstLoad":[a-z]*' | cut -d: -f2)
[ -z "$NEEDS_OWNER" ] && sleep 1
done

if [ "$NEEDS_OWNER" = "true" ]; then
    echo "[gef-bootstrap] Creando owner (POST /rest/owner/setup)..."
    OWNER_PAYLOAD=$(cat <<JSON
{"email":"${N8N_OWNER_EMAIL}", "firstName":"${N8N_OWNER_FIRST_NAME}",
"lastName":"${N8N_OWNER_LAST_NAME}", "password":"${N8N_OWNER_PASSWORD}"}
JSON
)
    wget -q -O- \
        --header="Content-Type: application/json" \
        --post-data="$OWNER_PAYLOAD" \
        http://localhost:5678/rest/owner/setup \
        >/dev/null
else
    echo "[gef-bootstrap] Owner ya existe (intento anterior incompleto), sigo."
fi

echo "[gef-bootstrap] Owner creado. Importando credenciales y workflows..."
n8n import:credentials --input="$CREDS_FILE"
n8n import:workflow --separate --input="$WORKFLOWS_DIR/"
publish_all

for f in "$WORKFLOWS_DIR"/*.json; do
    md5sum "$f" | cut -d ' ' -f1 > "$DATA_DIR/.hash-$(basename "$f").md5"
done

echo "[gef-bootstrap] Deteniendo instancia temporal para relanzar como proceso final..."
kill "$N8N_PID"
wait "$N8N_PID" 2>/dev/null || true
sleep 2

touch "$INIT_MARKER"
echo "[gef-bootstrap] Bootstrap inicial completo."
else
    echo "[gef-bootstrap] Volumen ya inicializado. Verificando cambios en workflows..."
    CHANGED=0
    for f in "$WORKFLOWS_DIR"/*.json; do
        HASH_FILE="$DATA_DIR/.hash-$(basename "$f").md5"
        NEW_HASH=$(md5sum "$f" | cut -d ' ' -f1)
        OLD_HASH=$(cat "$HASH_FILE" 2>/dev/null || echo "")
        if [ "$NEW_HASH" != "$OLD_HASH" ]; then
            echo "[gef-bootstrap] Cambio detectado en $(basename "$f"), reimportando..."
            # import:workflow, list:workflow y publish:workflow son operaciones
            # de CLI contra la base de datos -- no requieren el servidor HTTP
            # arriba, por eso pueden correr antes del exec final de mas abajo.
            n8n import:workflow --input="$f"
            echo "$NEW_HASH" > "$HASH_FILE"
            CHANGED=1
        fi
    done
    [ "$CHANGED" = "1" ] && publish_all
fi

echo "[gef-bootstrap] Arrancando n8n..."
exec n8n start

```

Por qué `publish:workflow` y no `update:workflow --all --active=true` — corregido tras probarlo contra una instancia real. La primera versión de este documento recomendaba `update:workflow --all --active=true` por ser el comando "clásico" y no requerir conocer el ID de antemano. Se levantó un n8n real (n8nio/n8n, versión 2.4.6) para probarlo, y el comando **fue rechazado**:

```
WARNING: The "update:workflow" command is deprecated.  
Workflow publishing via "update:workflow --all" is no longer supported.  
Please publish workflows individually using: publish:workflow --id=<workflow-id>
```

`publish:workflow --id=<ID>` sí funciona (confirmado: activa el workflow y queda reflejado en `n8n list:workflow`). Tampoco tiene flag `--all` (fue removido), así que `bootstrap.sh` resuelve el/los ID(s) con `n8n list:workflow | cut -d '|' -f1` y publica uno por uno — sigue sin requerir que nadie anote el ID a mano.

6. Integración Backend ↔ n8n

Se mantiene la decisión arquitectónica fijada (§3.3): el backend solo **dispara** scans hacia n8n; nunca recibe datos de vuelta por HTTP (n8n escribe directo a Postgres). El alcance de esta sección es exclusivamente la llamada backend→n8n.

Aspecto	Recomendación
URL interna	<code>http://n8n:5678/webhook/auditoria-automatizada</code> — nombre de servicio Docker, nunca <code>localhost</code> ni IP fija
Autenticación del webhook	Para el MVP, el webhook puede quedar sin autenticación adicional porque no es accesible fuera de <code>gef_net</code> (no se publica el puerto 5678 hacia afuera en producción). Si se expone públicamente, agregar <code>HTTPHeaderAuth</code> al nodo webhook con un secreto compartido en <code>.env</code> .
Manejo de errores	<code>N8nWebhookService</code> hoy hace <i>swallow</i> de la excepción (solo loggea). Debe relanzarla como una excepción de aplicación propia (p. ej. <code>N8nUnavailableException</code>) para que <code>AssetController</code> pueda devolver <code>502 Bad Gateway</code> en vez de <code>200 OK</code> cuando n8n no responde.
Timeouts	Configurar <code>RestClient</code> con <code>connectTimeout</code> ≈ 3s y <code>readTimeout</code> ≈ 5s. Sin timeout explícito, un n8n colgado deja el hilo (o virtual thread) del backend bloqueado indefinidamente.
Reintentos	1 reintento con backoff fijo de 2s es suficiente para un webhook de disparo (no es una operación idempotente-crítica; reintentar de más duplicaría el scan). No usar reintentos ilimitados.
Health check hacia n8n	El backend puede exponer su propio <code>/actuator/health</code> agregando un <code>HealthIndicator</code> custom que haga <code>GET http://n8n:5678/healthz</code> — útil para observabilidad, no crítico para el arranque (eso ya lo resuelve <code>depends_on</code> en Compose).
Payload	Corregir el payload actual (<code>{assetId, software, version, ecosystem}</code>) para incluir <code>activo_name</code> , que es el campo que el nodo <code>Edit Fields</code> del workflow real efectivamente lee (bug ya documentado como PB-056 en el análisis estratégico; se referencia aquí porque bloquea la demostración del pipeline automatizado, aunque su corrección de lógica de negocio excede el alcance de este documento).

7. Integración Frontend ↔ n8n

El frontend **nunca** debe llamar a n8n directamente ni abrir su UI (:5678) para que el usuario final vea el resultado de un scan. Todo pasa por la API del backend, que es quien conoce el estado real vía PostgreSQL (misma base que escribe n8n).

Flujo recomendado

1. Usuario hace click en "Disparar Scan" → `POST /api/assets/{id}/scan`.
2. Backend dispara el webhook a n8n y responde de inmediato (fire-and-forget desde la perspectiva HTTP; n8n corre de forma asíncrona).
3. El frontend inicia **polling corto** sobre `GET /api/assets/{id}` (intervalo 2–3s, máximo 30s) observando el campo `lastScan`: si su timestamp avanza respecto al valor previo al click, el scan terminó.
4. Al detectar el avance, el frontend refresca `GET /api/vulnerabilities?assetId={id}` para mostrar los hallazgos nuevos.
5. Si se cumple el timeout de 30s sin cambios, mostrar un estado "el scan sigue en curso, actualizá más tarde" en vez de un falso éxito (corrige el bug de UX descripto en el análisis estratégico, sección 3.5).

Decisión	Justificación
Polling en vez de WebSockets/SSE	El volumen de eventos es bajísimo (un scan cada tanto, disparado manualmente o 1 vez al día por el scheduler). WebSockets agregaría un canal persistente, gestión de reconexión y estado en el backend, sin beneficio medible en este MVP. Es la opción correcta según la propia guía de buenas prácticas: no diseñar para requisitos hipotéticos futuros.
Polling acotado (no infinito)	Evita seguir pegándole al backend indefinidamente si n8n falla o tarda; después de 30s se corta y se informa honestamente al usuario.
Comparación por <code>lastScan</code> , no por un endpoint de "estado de ejecución"	No requiere que n8n exponga nada nuevo; usa datos que el backend ya sirve hoy.

8. Variables de entorno (.env.example)

Archivo completo a crear en la raíz del repositorio. Ninguna de estas variables tiene un valor sensible por defecto embebido en `docker-compose.yml`; todas deben resolverse desde `.env` (que permanece fuera de git, ya cubierto por `.gitignore` existente).

Variable	Descripción	Ejemplo	Oblig.	Seguridad
<code>POSTGRES_DB</code>	Nombre de la base de datos de negocio	<code>security</code>	Sí	Baja
<code>POSTGRES_USER</code>	Usuario de PostgreSQL compartido por backend y n8n	<code>security_user</code>	Sí	Media
<code>POSTGRES_PASSWORD</code>	Password de ese usuario	<code>cambia-esto-por-una-clave-fuerte</code>	Sí	Alta — nunca commitear
<code>PGADMIN_DEFAULT_EMAIL</code>	Login de pgAdmin. Su validador de email rechaza dominios reservados como <code>.local</code> (verificado: "special-use or reserved name") — a diferencia de <code>N8N_OWNER_EMAIL</code> , acá no sirve un dominio <code>.local</code>	<code>admin@admin.com</code>	Sí	Media
<code>PGADMIN_DEFAULT_PASSWORD</code>	Password de pgAdmin	<code>cambia-esto-tambien</code>	Sí	Alta
<code>N8N_VERSION</code>	Versión de la imagen <code>n8nio/n8n</code> a fijar (build arg) — verificada contra <code>2.4.6</code> real	<code>2.4.6</code>	Sí	Baja
<code>N8N_ENCRYPTION_KEY</code>	Clave de cifrado de credenciales de n8n. Generar una sola vez con <code>openssl rand -base64 32</code> y no volver a cambiarla	<code>3F9a...</code> (32+ bytes aleatorios)	Sí	Crítica — si se pierde, las credenciales guardadas en el volumen quedan irrecuperables
<code>GENERIC_TIMEZONE</code>	Timezone del scheduler de n8n	<code>America/Argentina/Mendoza</code>	Sí	Baja
<code>N8N_HOST</code>	Host público/local donde vive n8n	<code>localhost</code>	Sí	Baja
<code>N8N_PROTOCOL</code>	Protocolo con el que n8n arma sus URLs	<code>http</code>	Sí	Baja

<code>N8N_WEBHOOK_PUBLIC_URL</code>	URL base que n8n usa para construir URLs de webhook mostradas al usuario	<code>http://localhost:5678/</code>	Sí	Baja
<code>N8N_OWNER_EMAIL</code>	Email del owner pre-provisionado	<code>admin@gef-secure.local</code>	Sí	Media
<code>N8N_OWNER_FIRST_NAME</code>	Nombre del owner	<code>GEF</code>	Sí	Baja
<code>N8N_OWNER_LAST_NAME</code>	Apellido del owner	<code>Secure</code>	Sí	Baja
<code>N8N_OWNER_PASSWORD</code>	Password del owner en texto plano — lo usa <code>bootstrap.sh</code> para llamar a <code>POST /rest/owner/setup</code> (no requiere hash, ver §4.2)	<code>cambia-esta-clave-tambien</code>	Sí	Alta — nunca commitear
<code>SLACK_BOT_TOKEN</code>	Token del bot de Slack usado por los nodos de notificación	<code>xoxb-...</code>	Sí (para notificaciones)	Alta
<code>GITHUB_TOKEN</code>	Personal Access Token de GitHub para la GitHub Advisory API (sube el rate limit de 60 a 5000 req/h)	<code>ghp_...</code>	Recomendada	Alta
<code>ALLOWED_ORIGINS</code>	Orígenes CORS permitidos por el backend	<code>http://localhost:3000</code> , <code>http://localhost:5173</code>	Sí	Baja

Ninguna variable marcada como seguridad **Alta** o **Crítica** debe tener un valor de ejemplo funcional en `.env.example` — solo placeholders descriptivos. El archivo real `.env` ya está cubierto por el `.gitignore` existente del proyecto (verificado: la entrada `.env` ya está presente).

9. Procedimiento completo de despliegue

```
1. git clone <url-del-repo>
2. cd GEF_Secure_App
3. cp .env.example .env
4. # Editar .env: contraseñas, N8N_ENCRYPTION_KEY, password del owner, tokens de Slack/GitHub
5. docker compose up -d
6. docker compose ps          # todos los servicios "healthy" o "running"
7. docker compose logs -f n8n # confirmar "[gef-bootstrap] Bootstrap inicial completo."
```

A partir del paso 5, sin ninguna acción manual adicional, quedan disponibles:

- PostgreSQL con las bases **security** y **n8n** creadas y con el esquema aplicado.
- Backend en <http://localhost:8081> (Swagger en </swagger-ui.html>).
- Frontend en <http://localhost:3000>.
- n8n en <http://localhost:5678> con el owner ya logueado por defecto, el workflow VMP importado y **activo**, y las 3 credenciales resueltas.

10. Validación del despliegue (checklist)

■	Verificación	Comando / cómo comprobarlo
■	Todos los contenedores healthy/running	<code>docker compose ps</code>
■	PostgreSQL acepta conexiones	<code>docker compose exec postgres pg_isready -U \$POSTGRES_USER</code>
■	Existen ambas bases de datos	<code>docker compose exec postgres psql -U \$POSTGRES_USER -l</code> → listar <code>security</code> y <code>n8n</code>
■	n8n responde y está listo	<code>curl -f http://localhost:5678/rest/settings</code>
■	El workflow fue importado	<code>docker compose exec n8n n8n list:workflow</code> → aparece "Pipeline de Gestión Automatizada de Vulnerabilidades (VMP)"
■	El workflow está activo	Mismo listado, columna <code>active</code> en <code>true</code>
■	Las 3 credenciales existen con los IDs correctos	<code>n8n</code> no tiene comando <code>list:credentials</code> (verificado en <code>n8n --help</code>) — consultar directo: <code>docker compose exec postgres psql -U \$POSTGRES_USER -d n8n -c "SELECT id,name,type FROM credentials_entity;"</code>
■	El backend levantó sin errores de conexión	<code>docker compose logs backend grep -i "started"</code>
■	El backend puede disparar un scan	<code>curl -X POST http://localhost:8081/api/assets/1/scan</code> → 200, y <code>docker compose logs n8n</code> muestra una ejecución nueva
■	El frontend carga y consume la API	Abrir <code>http://localhost:3000</code> , ver Dashboard con datos
■	Reiniciar el stack no reimporta ni duplica nada	<code>docker compose restart n8n</code> → logs muestran "Volumen ya inicializado", sin duplicar workflow ni credenciales
■	Un cambio en <code>workflows/*.json</code> se resincroniza solo	Editar el JSON, <code>docker compose restart n8n</code> , ver "Cambio detectado" en logs

11. Resolución de problemas

Síntoma	Causa probable	Diagnóstico	Solución
n8n reinicia en loop	<code>N8N_ENCRYPTION_KEY</code> cambió respecto al valor usado cuando se crearon las credenciales en el volumen	<code>docker compose logs n8n</code> — buscar error de desencriptado	Restaurar la clave original desde backup de <code>.env</code> , o borrar el volumen <code>n8n_data</code> y dejar que el bootstrap lo recree (se pierde historial de ejecuciones)
Backend nunca arranca / se queda esperando	n8n no llega a <code>healthy</code>	<code>docker compose ps</code> — columna STATUS de n8n	Revisar <code>docker compose logs n8n</code> ; suele ser fallo de conexión a Postgres (verificar <code>POSTGRES_*</code> en <code>.env</code>) o <code>bootstrap.sh</code> fallando por un JSON de workflow inválido
Workflow importado pero inactivo	<code>publish:workflow</code> corrió antes de que la importación terminara, o falló silenciosamente	<code>docker compose exec n8n n8n list:workflow</code>	<code>docker compose exec n8n n8n publish:workflow --id=<id></code> manualmente (el ID sale del comando anterior), luego revisar por qué el bootstrap no lo dejó publicado
El scan no dispara ninguna ejecución en n8n	URL de webhook incorrecta en el backend, o el nodo webhook del workflow tiene otro path	<code>docker compose logs backend grep n8n</code> + revisar el nodo Webhook en el JSON (<code>path</code>)	Confirmar que <code>N8N_WEBHOOK_URL</code> coincide exactamente con <code>http://n8n:5678/webhook/<path-del-nodo></code>
Los nodos de Slack no notifican	<code>SLACK_BOT_TOKEN</code> vacío o inválido	Ver ejecución fallida en n8n → nodo Slack → error de autenticación	Regenerar el token en Slack, actualizar <code>.env</code> , <code>docker compose restart n8n</code> (el overwrite se reconstruye en cada arranque)
GitHub Advisory API devuelve 403 / rate limited	<code>GITHUB_TOKEN</code> no configurado (límite público de 60 req/h)	Ver ejecución fallida en el nodo <code>Fetch_GitHub_Advisories</code>	Configurar <code>GITHUB_TOKEN</code> en <code>.env</code> y reiniciar n8n
Instalación limpia falla al crear el volumen de Postgres	Quedó un volumen <code>external</code> de una versión anterior del compose	<code>docker volume ls</code>	<code>docker compose down -v</code> seguido de <code>docker compose up -d</code> en un entorno de prueba limpio (nunca en un entorno con datos reales sin backup)

Falla el build del backend: <pre>./gradlew: not found</pre>	<code>backend/gradlew</code> tiene finales de línea CRLF (Windows) — rompe el shebang <code>#!/bin/sh</code> dentro del contenedor Linux (verificado con <code>file backend/gradlew</code>)	<code>file backend/gradlew</code> → si dice "with CRLF line terminators" es esto	<code>sed -i 's/\r\$//'</code> <code>backend/gradlew</code> una vez; considerar agregar <code>.gitattributes</code> con <code>gradlew text eol=lf</code> para que no vuelva a pasar en un clon en Windows
pgAdmin en crash-loop (<code>Restarting</code> constante)	<code>PGADMIN_DEFAULT_EMAIL</code> con un dominio reservado (p. ej. <code>.local</code>) — el validador de pgAdmin lo rechaza (verificado)	<code>docker compose logs pgadmin</code> → "does not appear to be a valid email address... special-use or reserved name"	Usar un dominio no reservado en <code>PGADMIN_DEFAULT_EMAIL</code> (p. ej. <code>admin@admin.com</code>), luego <code>docker compose up -d pgadmin</code>

12. Mejoras recomendadas

Automatización del despliegue

- Agregar un script `smoke-test.sh` (ya previsto como PB-008 en el análisis estratégico) que ejecute el checklist de la sección 10 de forma automática y falle el pipeline de CI si algo no responde.
- Publicar la imagen custom de n8n (con el bootstrap ya embebido) en un registro propio versionado por tag de git, para no reconstruirla en cada `docker compose up` en CI.

Escalabilidad

- El diseño actual asume una sola instancia de n8n (modo `main`). Si el volumen de scans crece, migrar a modo `queue` con Redis es el camino documentado por n8n — a partir de ahí, `CREDENTIALS_OVERWRITE_DATA` debe estar presente en **todos** los workers, no solo en el proceso principal.

Seguridad

- Migrar de `CREDENTIALS_OVERWRITE_DATA` (variable de entorno plana) a `CREDENTIALS_OVERWRITE_FILE` apuntando a un secreto gestionado (Docker secret o equivalente) antes de cualquier despliegue fuera del entorno de tesis/demo.
- No publicar el puerto `5678` de n8n hacia afuera del host en un despliegue real; solo el frontend y, si hace falta, el backend.
- Rotar `N8N_OWNER_PASSWORD` y todos los tokens antes de cualquier demostración pública del repositorio.

Mantenibilidad

- Dejar anotado en el `README.md` del proyecto que `N8N_INSTANCE_OWNER_MANAGED_BY_ENV` requiere licencia paga y se ignora en silencio en Community Edition (sección 4.2) — para que nadie pierda tiempo "arreglando" el bootstrap reemplazando el mecanismo de `POST /rest/owner/setup` por esa variable pensando que es una mejora.
- Como ya señala el análisis estratégico (MEJ-06), la lógica del *Risk Assessment Engine* embebida en un Code node de n8n sigue sin ser testeable ni versionable de forma legible; esto es independiente de la automatización de infraestructura pero comparte el mismo objetivo de reproducibilidad.

Versionado de workflows

- El mecanismo de hash-check (§4.3) es suficiente para un único workflow. Si el proyecto crece a múltiples workflows con dependencias entre sí, evaluar migrar a n8n Source Control nativo (requiere licencia) para obtener diffs visuales y despliegue por entorno (dev/staging/prod) desde la propia UI de n8n.

Integración backend↔n8n

- Una vez corregido el payload (§6), agregar un test de integración en el backend que levante un stub HTTP en el puerto de n8n y verifique que `N8nWebhookService` envía el campo `activo_name` esperado — cierra la brecha de "cero tests de negocio" señalada en el análisis estratégico, específicamente para el punto de integración que este documento automatiza.

Organización general del proyecto

- Los nuevos artefactos de esta propuesta (`n8n/Dockerfile`, `n8n/bootstrap.sh`, `n8n-provisioning/credentials.json`) se agrupan en carpetas propias en la raíz del repo, siguiendo el mismo patrón que ya usan `backend/`, `frontend/` e `init/` — no se mezclan con el código de aplicación.

13. Conclusiones finales

El proyecto ya tiene, hoy, casi todas las piezas necesarias para la automatización completa: un workflow versionado en git con IDs de credenciales estables, un esquema de base de datos que ya separa `security` de `n8n`, y healthchecks parciales en el compose. Lo que falta no es infraestructura nueva sino **conectar correctamente lo que ya existe**: un punto de entrada de n8n propio que reemplace la configuración manual por variables de entorno y comandos de CLI ya soportados oficialmente.

Este documento corrige además una brecha real del documento estratégico: dos de sus recomendaciones para n8n (`N8N_PUBLIC_API_ENABLED` y el aprovisionamiento de credenciales vía API Key) no son ejecutables sin un paso manual previo en la versión actual de n8n. La solución aquí propuesta —`import:credentials` + `CREDENTIALS_OVERWRITE_DATA` + hash-check de workflows en el entrypoint— fue verificada contra la documentación oficial de n8n y no depende de ningún paso interactivo en ningún punto del ciclo de vida del contenedor.

Con los cambios de las secciones 5 a 8 aplicados, el flujo `git clone → cp .env.example .env → docker compose up -d` queda cumplido en su totalidad para la capa de infraestructura. La corrección de los bugs de lógica de negocio del pipeline (payload, `ON CONFLICT`, SQL injection en nodos, MTTR) sigue siendo responsabilidad del roadmap ya definido en [GEF_Secure_Analisis_Estrategico_MVP.pdf](#), y no bloquea que la infraestructura arranque sola — separación de responsabilidades que además facilita probar cada capa de forma independiente.

Estado al aplicar este documento: instalación reproducible, sin pasos manuales en n8n, con la misma robustez de arranque que ya tenía Postgres — extendida ahora a n8n y a la cadena completa de dependencias del stack.

14. Apéndice: Contrato de scan por alcance (activo / entorno / global)

Estado: trabajo futuro, no implementado todavía. Esta sección no forma parte del alcance de automatización de infraestructura de las secciones 1 a 13 — se agrega como referencia para la **próxima etapa de implementación** (scan por entorno y scan global a demanda), de modo que el contrato quede fijado *antes* de escribir el código nuevo y no se repita el mismo tipo de bug que hoy afecta al scan por activo (el campo que envía `N8nWebhookService` no coincide con el campo que el workflow espera leer).

14.1 El workflow ya soporta los 3 casos — evidencia en el JSON

Inspeccionando directamente los nodos `Edit Fields` y `Switch1` del archivo `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json`, se confirma que el ruteo por alcance **ya está implementado dentro de n8n**, sobre el mismo webhook único (`/webhook/auditoria-automatizada`). No hace falta construirlo — hace falta que el backend le hable en el idioma correcto.

Nodo `Edit Fields` (tipo `set`) arma dos variables a partir del body del webhook:

```
filtro_entorno = {{ Webhook.isExecuted ? Webhook.body.entorno      : "TODOS" }}
filtro_activo  = {{ Webhook.isExecuted ? Webhook.body.activo_name  : "TODOS" }}
```

Nodo `Switch1` clasifica el scan a partir de esas dos variables:

```
filtro_activo != 'TODOS' → rama ACTIVO
filtro_entorno != 'TODOS' → rama ENTORNO    (solo si filtro_activo == 'TODOS')
en cualquier otro caso   → rama GLOBAL
```

Esta clasificación es la que después decide, entre otras cosas, cuál de los 3 nodos de notificación Slack dispara (el análisis estratégico ya identificó esta arquitectura de notificación por tipo de scan como una fortaleza del proyecto).

14.2 El bug pendiente que hay que tener en cuenta (BUG-N8N-07)

El fallback a `"TODOS"` en `Edit Fields` sólo se aplica cuando el workflow **no** fue disparado por webhook (es decir, cuando dispara el scheduler). Si lo dispara el webhook pero el body no trae uno de los dos campos, ese campo queda `undefined` en vez de `"TODOS"` — esto es exactamente el **BUG-N8N-07** que [GEF_Secure_Analisis_Estrategico_MVP.pdf](#) ya documenta con prioridad CRÍTICA (Iteración 1 del roadmap). La solución de fondo (agregar el fallback también dentro de la rama del webhook) sigue siendo responsabilidad de ese roadmap. La sección 14.3 da una forma de evitar pisar este bug incluso antes de que se corrija.

14.3 Contrato de payload recomendado

Mientras el bug de 14.2 no esté corregido, la forma segura de no pisarlo es que el backend **mande siempre los dos campos explícitamente**, usando el literal `"TODOS"` para el que no aplique al caso:

Caso	Trigger	Body que debe enviar el backend	Endpoint backend (a crear)
------	---------	---------------------------------	----------------------------

Scan por activo	Ya implementado hoy	<code>{"activo_name": "react", "entorno": "TODO"}"</code>	<code>POST /api/assets/{id}/scan</code> (ya existe)
Scan por entorno	Pendiente	<code>{"activo_name": "TODO", "entorno": "Producción"}"</code>	<code>POST /api/environments/{id}/scan</code> (PB-061)
Scan global	Pendiente (manual). El scheduler diario ya cubre el caso automático	<code>{"activo_name": "TODO", "entorno": "TODO"}"</code>	<code>POST /api/scan</code> (PB-062)

Con esta convención, los 3 casos funcionan **incluso sin corregir BUG-N8N-07**, porque ningún campo llega nunca vacío desde el backend. Corregir el bug en n8n igual sigue siendo recomendable como red de seguridad para otros llamadores (pruebas manuales con `curl`, Postman, etc.).

14.4 Qué falta construir (fuera del alcance de este documento)

Capa	Trabajo pendiente
Backend	Nuevos métodos en <code>N8nWebhookService</code> (p. ej. <code>triggerScanByEnvironment</code> , <code>triggerScanGlobal</code>) que respeten el contrato de 14.3. Nuevos endpoints <code>POST /api/environments/{id}/scan</code> (PB-061) y <code>POST /api/scan</code> (PB-062). Aprovechar el cambio para corregir a la vez el payload del scan por activo (bug ya documentado en la sección 6 de este mismo documento).
Backend (lectura)	El polling descrito en la sección 7 compara <code>asset.lastScan</code> , lo cual sólo sirve para el caso ACTIVO. Para ENTORNO y GLOBAL no hay un único activo que mirar — se necesita un endpoint de lectura, por ejemplo <code>GET /api/scan-reports/latest?targetType=ENTORNO&targetName=Producción</code> , que exponga el <code>scan_reports</code> más reciente para ese alcance (la tabla ya tiene las columnas <code>target_type</code> y <code>target_name</code>).
Frontend	Puntos de disparo para "escanear entorno completo" (en la vista de Entornos) y "escanear todo" (en Dashboard o Configuración). Mismo patrón de polling acotado a 30s de la sección 7, pero comparando el timestamp del <code>scan_reports</code> más reciente en vez de <code>lastScan</code> de un activo puntual.
n8n	Corregir BUG-N8N-07 (fallback <code> 'TODO'</code> también dentro de la rama del webhook) — ya priorizado como CRÍTICA en el roadmap del análisis estratégico.

Nada de esto requiere cambios en Docker ni en el bootstrap de n8n (secciones 4 y 5) — es enteramente trabajo de lógica de negocio en backend y frontend.